

Manual Técnico: Analizador Léxico-Sintáctico Wison

****Versión:**** 1.0

****Fecha:**** Abril 2026

****Proyecto:**** Compilador de Lenguaje Wison

****Descripción:**** Sistema completo de análisis léxico y sintáctico para el lenguaje de definición de gramáticas Wison

Tabla de Contenidos

Manual Técnico: Analizador Léxico-Sintáctico Wison	1
Tabla de Contenidos	2
Introducción General	2
Descripción del Proyecto	3
Arquitectura del Sistema	4
Stack Tecnológico	5
Lenguaje Wison: Especificación	6
Componentes del Frontend	11
Componentes del Backend	13
Algoritmos Principales	29
Flujo de Datos Completo	40
Limitaciones y Consideraciones	45
Guía de Instalación	47
Ejemplos de Uso	53
Conclusión	58

Introducción General

Propósito

Este proyecto implementa un **analizador léxico-sintáctico completo** para el lenguaje **Wison**, un lenguaje de definición de gramáticas diseñado específicamente para análisis educativo. El sistema permite a los usuarios:

- **Definir gramáticas personalizadas** usando una sintaxis intuitiva
- **Analizar cadenas de entrada** contra esas gramáticas
- **Visualizar el árbol de derivación** de forma clara y legible
- **Recibir retroalimentación detallada** sobre errores en gramática, léxica y sintáctica

Características Clave

****Meta-Análisis****: El sistema analiza definiciones de Wison usando Jison

****Análisis Léxico Dinámico****: Genera analizadores léxicos automáticamente desde definiciones

****Parser LL(1) Completo****: Implementación predictiva de tabla-dirigida

****Visualización de Árboles****: Representación ASCII clara del árbol de derivación

****Validación Completa****: Detección de recursión izquierda, conflictos FIRST/FIRST, uso de símbolos

****Mensajes en Español****: Todos los errores en español para mejor comprensión

****Encoding UTF-8****: Soporte completo para caracteres especiales y acentos

****Interfaz Web Moderna****: Angular 21 con Bootstrap 5 para mejor UX

Descripción del Proyecto

Problema que Resuelve

En educación sobre compiladores, es frecuente que los estudiantes necesiten:

1. Probar si una gramática es válida (sin recursión izquierda, sin conflictos)
2. Analizar cadenas de entrada contra esas gramáticas
3. Entender el proceso de análisis viendo el árbol de derivación
4. Obtener mensajes de error claros en su idioma

****Solución****: Wison proporciona una plataforma integrada donde cualquiera puede definir una gramática en minutos y analizar cadenas inmediatamente.

Diferenciadores

| Aspecto | Wison | Genéricos |

|-----|-----|-----|

| Sintaxis Grammar | Diseñada para claridad | Confusa para principiantes |

| Validación | Completa (recursión, FIRST) | Parcial o ninguna |

| Errores | Español + contexto | Inglés, genéricos |

| Árbol de Derivación | Visualización ASCII | Texto crudo o necesita herramientas externas |

| Encoding | UTF-8 garantizado | Puede variar |

Arquitectura del Sistema

Componentes Principales

****Nivel 1: Interfaz de Usuario****

- Angular SPA (Single Page Application)
- Bootstrap 5 para styling responsivo
- Textarea para gramática, campo de entrada, botón de análisis

****Nivel 2: API HTTP****

- Express.js servidor REST
- CORS habilitado para desarrollo local
- Rutas: `/analyze`, `/tokenize`, `/generate`, `/health`

****Nivel 3: Procesamiento de Gramáticas****

- Jison meta-parser para analizar definiciones Wison
- Validadores semánticos recursivos
- Extractores de estructura (terminales, no-terminales, producciones)

****Nivel 4: Análisis Léxico****

- Compilador dinámico de expresiones terminales
- Tokenizador con búsqueda greedy de coincidencia más larga
- Manejo de errores léxicos

****Nivel 5: Análisis Sintáctico****

- Parser LL(1) predictivo
- Construcción simultánea del árbol de derivación
- Cálculo de conjuntos FIRST
- Registro de pasos de análisis

Stack Tecnológico

Frontend

Tecnología	Versión	Propósito
-----	-----	-----
Angular	21.2.0	Framework de aplicación web
TypeScript	5.9.2	Lenguaje tipado para JS
Bootstrap	5.x (CDN)	Framework CSS responsivo
RxJS	7.8.0	Programación reactiva
Angular CLI	21.2.6	Herramienta de construcción

****Navegadores soportados****: Chrome, Edge, Firefox (modernos, ES2020+)

Backend

Tecnología	Versión	Propósito
-----	-----	-----
Node.js	18+ LTS	Runtime JavaScript
Express	5.2.1	Framework HTTP
Jison	0.4.18	Meta-parser (genera parsers desde gramáticas)
CORS	2.8.6	Middleware de origen cruzado

Requisitos del sistema: Node.js 18 LTS o superior

Desarrollo

Herramienta	Propósito
-----	-----
npm	Gestor de dependencias
git	Control de versiones
Visual Studio Code	Editor recomendado
.editorconfig	Configuración de encoding UTF-8 sin BOM

Lenguaje Wison: Especificación

Descripción General

Wison es un lenguaje de definición de gramáticas libre de contexto (context-free), diseñado para ser intuitivo y educativo. Define tres componentes principales:

1. **Bloque Lex**: Definición de terminales (tokens)

2. ****Bloque Syntax****: Definición de no-terminales y producciones
3. ****Marcadores****: Símbolos iniciales y finales

Sintaxis Completa

Estructura General

...

Wison₂

```
Lex {
    [Terminal definitions]
}

Syntax {
    [Non-terminal definitions]
}
```

?Wison

...

O alternativa (equivalente):

...

Wison

```
Lex {{
    [Terminal definitions]
}}

Syntax {{
    [Non-terminal definitions]
```

}}

FIN

Bloque Lex: Definición de Terminales

Terminal \$_NAME <- EXPRESSION ;

****Componentes:****

- `Terminal`: Palabra clave obligatoria
- `\$\_NAME`: Nombre del terminal (comienza con `\$\_`, sin espacios)
- `<-`: Operador de asignación
- `EXPRESSION`: Expresión regular en notación Wison
- `;`: Terminador obligatorio

****Ejemplos de expresiones:****

Terminal \$_plus <- '+' ; // Carácter literal

Terminal \$_digit <- '[0-9]' ; // Clase de caracteres

Terminal \$_id <- '[a-zA-Z][a-zA-Z0-9]*' ; // Identificador

Terminal \$_space <- '[\t]' ; // Espacio en blanco

Terminal \$_num <- '[0-9]+' ; // Uno o más dígitos

Terminal \$_letter <- '[a-z]' ; // Alternancia implícita

****Operadores soportados:****

| Operador | Significado | Ejemplo |

|-----|-----|-----|
| `[abc]` | Clase de caracteres | `[aeiou]` |
| `[a-z]` | Rango de caracteres | `[0-9]` |
| `\*` | Cero o más | `ab\*` |
| `+` | Uno o más | `a+` |
| `?` | Cero o uno | `x?` |
| `()` | Agrupación | `(ab)+` |
| `""` | Literal exacto | `+` |
| `\$\_REF` | Referencia a otro terminal | `\$\_digit` |

Bloque Syntax: Definición de No-Terminales

No_Terminal %_NAME ;
Initial_Sim %_START ;
%_NAME <= PRODUCTION | PRODUCTION | ... ;

****Tipos de líneas:****

1. ****Declaración de No-Terminal (opcional, pero recomendado)****

No_Terminal %_Expr ;
No_Terminal %_Term ;

2. ****Símbolo Inicial (obligatorio, único)****

Initial_Sim %_Expr ;

3. ****Producciones (al menos una)****

`%_Expr <= %_Term | %_Expr '+' %_Term ;`

`%_Term <= $_digit | '(' %_Expr ')';`

****Convenciones de nombres:****

- No-terminales: ``%_`` prefix (ej: ``%_Expr``, ``%_Term``)

- Terminales: ``$_`` prefix (ej: ``$_plus``, ``$_digit``)

- Literales: entre comillas simples (ej: ``+'``, ``)'``)

Comentarios

Los comentarios se soportan en ambos bloques:

Terminal `$_id <- '[a-zA-Z]+' ;` // Comentario de línea

`/* Comentario`

`de bloque`

`multilínea */`

Terminal `$_num <- '[0-9]+' ;`

****Formatos:****

- ``//`` para comentarios hasta fin de línea

- ``/* ... */`` para comentarios de bloque multilínea

Restricciones Gramaticales

1. ****Sin producciones epsilon (vacías)****

`%_Expr <= %_Term | ;` // ERROR: alternativa vacía

`%_Expr <= %_Term ;` // Correcto

2. ****Sin recursión izquierda****

```
%_Expr <= %_Expr '+' %_Term | %_Term ; // ERROR: A -> A...
```

```
%_Expr <= %_Term '+' %_Expr | %_Term ; // Correcto (recursión derecha)
```

3. ****Todos los símbolos usados deben estar declarados****

```
%_Expr <= %_Undefined ; // ERROR: no declarado
```

```
%_Expr <= %_Term ; // Correcto, %_Term está declarado
```

4. ****Símbolo inicial debe estar declarado****

```
Initial_Sim %_NonExistent ; // ERROR: no existe
```

```
Initial_Sim %_Expr ; // Correcto
```

Componentes del Frontend

Estructura de Directorios

frontend/

├── src/

| ├── index.html // Punto de entrada HTML

| ├── styles.css // Estilos globales

| ├── app/

| | ├── app.ts // Componente principal

| | ├── app.html // Template

| | ├── app.css // Estilos del componente

| | ├── app.config.ts // Configuración de Angular

| | └── app.spec.ts // Tests

| └── main.ts // Bootstrap de la aplicación

└── proxy.conf.json // Configuración proxy para desarrollo

```
|— angular.json      // Configuración de Angular CLI
|— tsconfig.json     // Configuración de TypeScript
└— package.json      // Dependencias de npm
```

Componente Principal: app.ts

Responsabilidades

```
``typescript
// Inyecciones de dependencia
- HttpClient: Para comunicación HTTP con backend
- ChangeDetectorRef: Para forzar detección de cambios en async
``
```

Propiedades del Componente

```
``typescript
// Estado de UI
source: string    // Código Wison en textarea
input: string     // Cadena a analizar
loading: boolean  // Estado de botón "Analizando..."
message: string   // Mensaje de estado
errors: string[]  // Lista de errores actuales

// Resultados del análisis
grammar: any      // Objeto de estructura de gramática
lexical: any      // Resultados de análisis léxico
```

syntax: any // Resultados de análisis sintáctico

treeRoot: DerivationNode | null // Raíz del árbol de derivación

Métodos Principales

****analyze(): void****

- Gatillado por click del botón "Analizar"
- Valida que source y input no estén vacíos
- Realiza POST a `/analyze` con ambos parámetros
- Timeout: 15 segundos
- Maneja respuestas exitosas (200, 201) e errores (4xx, 5xx)
- Llama a `detectChanges()` para UI update

****treeToAscii(): string****

- Convierte `DerivationNode` tree a representación ASCII
- Usa prefijos de indentación y conectores de rama (|-, \-)
- Incluye lexemas en etiquetas cuando aplica
- Algoritmo recursivo con tracking de profundidad

Componentes del Backend

Estructura de Directorios

...

backend/

├── src/

| ├── index.js // Servidor Express principal

| ├── routes/

| | └── analyze.routes.js // Rutas de análisis

| └── controllers/

```

| | └─ analyze.controller.js    // Handler del análisis
| | └─ tokenize.controller.js  // Handler de tokenización
| | └─ generate.controller.js  // Handler de validación
| └─ services/
| | └─ lexer.service.js        // Compilador y tokenizador léxico
| | └─ ll-parser.service.js    // Parser LL(1)
| └─ parsers/
| | └─ wison.configuration.json // Meta-parser Jison
| | └─ wison.parser.js         // Extractor de estructura
| | └─ wison.validators.js     // Validadores de gramática
| └─ utils/
|   └─ error-messages.js      // Normalización de errores
└─ package.json
  └─ package-lock.json
...

```

Punto de Entrada: index.js

****Puerto**:** 3000 (por defecto, configurable via `PORT env`)

****Responsabilidades****

- Iniciar servidor Express
- Configurar CORS
- Montar rutas
- Servir healthcheck

```
```)javascript

import express from 'express';

import cors from 'cors';

import analyzeRoutes from './routes/analyze.routes.js';

const app = express();

const PORT = process.env.PORT || 3000;

// Middleware
app.use(
 cors({
 origin: ['http://localhost:4200', 'http://localhost:3000'],
 credentials: true
 })
);

app.use(express.json({ limit: '1mb' }));

// Routes
app.use('/', analyzeRoutes);

// Health check
app.get('/health', (req, res) => {
 res.json({ ok: true, service: 'wison-backend' });
});
```

```
// Error handling
app.use((err, req, res, next) => {
 console.error('Error:', err);
 res.status(500).json({
 ok: false,
 error: err.message || 'Error interno del servidor'
 });
});

app.listen(PORT, () => {
 console.log(`Wison Backend ejecutándose en puerto ${PORT}`);
});

```

#### Rutas: analyze.routes.js

```
```.javascript
import express from 'express';

import { analyzeInput } from '../controllers/analyze.controller.js';

import { tokenizeInput as tokenizeInputHandler } from
'../controllers/tokenize.controller.js';

import { generateGrammar } from '../controllers/generate.controller.js';

const router = express.Router();

// POST /analyze - Análisis completo (léxico + sintáctico)
router.post('/analyze', analyzeInput);

```



```
// POST /tokenize - Solo análisis léxico
router.post('/tokenize', tokenizeInputHandler);

// POST /generate - Validación de gramática
router.post('/generate', generateGrammar);

export default router;
...

```

Controlador: [analyze.controller.js](#)

****Flujo:****

1. Valida entrada (source no vacío, input es string)
2. Parsea gramática Wison
3. Si hay error: retorna 400 con grammarErrors
4. Tokeniza entrada
5. Si hay error: retorna 400 con lexicalErrors
6. Parsea tokens con LL(1)
7. Retorna resultado completo (grammar, lexical, syntax)

javascript

```
import { parseWisonWithJison } from '../parsers/wison.parser.js';
import { tokenizeInput } from '../services/lexer.service.js';
import { parseTokensWithLL } from '../services/ll-parser.service.js';
import { normalizeErrorMessage } from '../utils/error-messages.js';

```

```
export async function analyzeInput(req, res) {  
  try {  
    const { source, input } = req.body;  
  
    // Validación  
    if (!source || typeof source !== 'string' || !source.trim()) {  
      return res.status(400).json({  
        ok: false,  
        grammarErrors: ['La gramática Wison es obligatoria']  
      });  
    }  
  
    if (typeof input !== 'string') {  
      return res.status(400).json({  
        ok: false,  
        lexicalErrors: ['La cadena de entrada debe ser un string']  
      });  
    }  
  
    // Fase 1: Análisis de gramática  
    const grammar = parseWisonWithJison(source);  
  
    if (!grammar.ok) {  
      return res.status(400).json({  
        ok: false,
```

```

        grammarErrors: grammar.errors
    });
}

// Fase 2: Análisis léxico

const lexical = tokenizeInput(input, grammar.terminals);

if (!lexical.ok) {
    return res.status(400).json({
        ok: false,
        lexicalErrors: lexical.errors,
        syntaxMessage: 'No se puede analizar sintácticamente sin tokens válidos'
    });
}

// Fase 3: Análisis sintáctico

const syntax = parseTokensWithLL(lexical.tokens, grammar);

res.json({
    ok: true,
    grammar,
    lexical,
    syntax
});
} catch (error) {
    console.error('Error en analyzeInput:', error);
}

```

```
const message = normalizeErrorMessage(error);

res.status(500).json({
  ok: false,
  error: message
});
}
}
...

```

Meta-Parser Jison: [wison.configuration.jison](#)

****Propósito**:** Analizar definiciones Wison y extraer bloques Lex y Syntax crudos

****Estados del Lexer:****

- `INITIAL`: Top level (palabras clave Wison, Lex, Syntax)
- `IN\_LEX`: Dentro del bloque Lex
- `IN\_SYNTAX`: Dentro del bloque Syntax

```
``jison
```

```
%lex
```

```
/- Estados -/
```

```
%s IN_LEX
```

```
%s IN_SYNTAX
```

```
%%
```

```

<INITIAL>{
    "Wisonζ"    return 'WISON';
    "?Wison"    return 'END_WISON';
    "Wison"     return 'WISON';
    "FIN"       return 'END_WISON';
    "Lex"       this.begin('IN_LEX'); return 'LEX';
    "Syntax"    this.begin('IN_SYNTAX'); return 'SYNTAX';
    "{"         return 'OPEN_BLOCK';
    "}"         return 'CLOSE_BLOCK';
    "{{" | "{"  return 'OPEN_BLOCK';
    "}}" | "}"  return 'CLOSE_BLOCK';
    "#".*       /* Ignora comentarios */
    "/\*"["\s\S"]?*"\*/" /* Ignora comentarios de bloque */
    \s+         /* Ignora espacios */
}

```

```

<IN_LEX>{
    "}"         this.popState(); return 'CLOSE_BLOCK';
    "}}"        this.popState(); return 'CLOSE_BLOCK';
    "}"         this.popState(); return 'CLOSE_BLOCK';
    [^]]+       return 'LEX_CHAR';
    "#".*       /* Ignora comentarios */
    "/\*"["\s\S"]?*"\*/" /* Ignora comentarios */
}

```

```

<IN_SYNTAX>{
    "}"      this.popState(); return 'CLOSE_BLOCK';
    "}}"     this.popState(); return 'CLOSE_BLOCK';
    "}"      this.popState(); return 'CLOSE_BLOCK';
    [^]+     return 'SYNTAX_CHAR';
    "#".*    /* Ignora comentarios */
    "/\*"["\s\S]*?"*\\" /* Ignora comentarios */
}

```

```

<<EOF>>    return 'EOF';
.           return 'INVALID';

```

```

/lex

```

```

%start program

```

```

%%

```

```

program

```

```

: WISON LEX OPEN_BLOCK lexBlock CLOSE_BLOCK SYNTAX OPEN_BLOCK syntaxBlock
CLOSE_BLOCK END_WISON

```

```

{ return { lexRaw: $4, syntaxRaw: $8 }; }

```

```

| WISON LEX OPEN_BLOCK lexBlock CLOSE_BLOCK SYNTAX OPEN_BLOCK syntaxBlock
CLOSE_BLOCK 'FIN'

```

```

{ return { lexRaw: $4, syntaxRaw: $8 }; }

```

```

;

```

lexBlock

```
: lexBlock LEX_CHAR { $$ = $1 + $2; }  
| LEX_CHAR { $$ = $1; }  
| /* empty */ { $$ = ""; }  
;
```

syntaxBlock

```
: syntaxBlock SYNTAX_CHAR { $$ = $1 + $2; }  
| SYNTAX_CHAR { $$ = $1; }  
| /* empty */ { $$ = ""; }  
;
```

%%

...

[Extractor de Estructura: wison.parser.js](#)

****Funciones principales:****

``javascript

// Extrae terminales de bloque Lex

```
function extractTerminals(lexRaw) {
```

```
    const terminals = [];
```

```
    const regex = /Terminal\s+\$_(\w+)\s*<-\s*(.+?)\s*/g;
```

```
    let match;
```

```
    while ((match = regex.exec(lexRaw)) !== null) {
```

```

    terminals.push({
      name: `_${match[1]}`,
      expression: match[2].trim()
    });
  }
  return terminals;
}

```

```

// Extrae no-terminales de bloque Syntax
function extractNonTerminals(syntaxRaw) {
  const nonTerminals = [];
  const regex = /No_Terminal\s+%_(\w+)\s*/g;
  let match;
  while ((match = regex.exec(syntaxRaw)) !== null) {
    nonTerminals.push(`_${match[1]}`);
  }
  return nonTerminals;
}

```

```

// Extrae símbolo inicial
function extractStartSymbol(syntaxRaw) {
  const regex = /Initial_Sim\s+(%\_\w+)\s*/;
  const match = regex.exec(syntaxRaw);
  return match ? match[1] : null;
}

```



```

// Extrae producciones

function extractProductions(syntaxRaw) {

  const productions = {};

  const regex = /(%_\w+)\s*<=\s*(.+?)\s*/g;

  let match;

  while ((match = regex.exec(syntaxRaw)) !== null) {

    const nonTerminal = match[1];

    const alternatives = match[2]

      .split('|')

      .map(alt => alt.trim());

    if (!productions[nonTerminal]) {

      productions[nonTerminal] = [];

    }

    productions[nonTerminal].push(...alternatives);

  }

  return productions;

}

// Orquestador principal

export function parseWisonWithJison(source) {

  try {

    const parsed = blockParser.parse(source);

    const terminals = extractTerminals(parsed.lexRaw);

    const nonTerminals = extractNonTerminals(parsed.syntaxRaw);

    const startSymbol = extractStartSymbol(parsed.syntaxRaw);

  }

}

```

```
const productions = extractProductions(parsed.syntaxRaw);
```

```
const validation = validateWilsonGrammar({  
  terminals,  
  nonTerminals,  
  productions,  
  startSymbol,  
  lexRaw: parsed.lexRaw,  
  syntaxRaw: parsed.syntaxRaw  
});
```

```
return {  
  ok: validation.errors.length === 0,  
  terminals,  
  nonTerminals,  
  productions,  
  startSymbol: startSymbol || nonTerminals[0],  
  errors: validation.errors,  
  raw: {  
    lex: parsed.lexRaw,  
    syntax: parsed.syntaxRaw  
  }  
};  
} catch (error) {  
  return {  
    ok: false,
```

```
    terminals: [],  
    nonTerminals: [],  
    productions: {},  
    startSymbol: null,  
    errors: [normalizeErrorMessage(error)]  
  };  
}  
}  
...
```

Validadores: [wison.validators.js](#)

****Validaciones implementadas:****

1. **Formato de líneas (sintaxis)**

```
``javascript  
validateLexLineSyntax(lexRaw)  
validateSyntaxLineSyntax(syntaxRaw)  
...
```

2. **Uso de símbolos**

```
``javascript  
validateTerminalUsage(terminals, productions)  
validateNonTerminalUsage(nonTerminals, productions)  
...
```

3. ****Símbolo inicial****

```
``javascript
validateInitialSymbol(startSymbol, nonTerminals)
``
```

4. ****Sin producciones vacías****

```
```\njavascript\nvalidateNoEpsilonProductions(productions)\n```\n
```

## 5. **\*\*Detección de recursión izquierda\*\***

```
```javascript
detectLeftRecursion(nonTerminal, productions, visited, recursionStack) {
    // DFS-based cycle detection
    // Returns true if A =>* A found
}
```
```

## 6. \*\*Detección de conflictos FIRST/FIRST\*\*

```
```javascript
validatePredictiveConflicts(nonTerminal, productions) {
    // Verifica que no haya dos alternativas que comiencen
    // con el mismo terminal (FIRST/FIRST conflict)
}
```
```

## 7. **\*\*Cálculo de conjuntos FIRST\*\***

```
```\n\nbuildFirstSets(terminals, nonTerminals, productions) {\n\n    // Fixed-point iteration hasta convergencia\n\n    // Returns { nonTerminal: [first_symbols...] }\n\n}\n\n```\n\n---
```

Algoritmos Principales

I. Análisis Léxico Dinámico

1. *Tokenización de Expresiones*

****Entrada****: Cadena con expresión Wison

****Salida****: Array de tokens parsed

****Algoritmo****:

```\n\n

Para cada carácter en expresión:

Si es '{' o '[':

Parse clase o grupo recursivamente

Si es '\*', '+', '?':

Mark anterior como modificado

Si es '':

Parse literal entre comillas

Si es mayúscula (terminal ref):

Acumula nombre de terminal

Si es '|':

Crea alternancia

...

**\*\*Ejemplo:\*\***

...

Entrada: '[a-zA-Z][0-9]\*'

Output: [

{ type: 'class', chars: ['a-z', 'A-Z'] },

{ type: 'class\_star', chars: ['0-9'] }

]

...

## ***2. Compilación de Expresiones a Regex***

**\*\*Entrada\*\***: Tokens de expresión parsed

**\*\*Salida\*\***: String de expresión regular JavaScript válida

**\*\*Algoritmo:\*\***

...

Para cada token:

Si es literal: escapa caracteres especiales

Si es clase: convierte a [...]

Si es modificador (\*/+/?): aplica a anterior

Si es alternancia: separa con |

Si es referencia: resuelve terminal recursivamente

...

**\*\*Ejemplo:\*\***

...

Entrada: Terminal `$_id <- '[a-zA-Z][a-zA-Z0-9]*'` ;

Output: `regex = /^[a-zA-Z][a-zA-Z0-9]*/`

...

### **3. Tokenización de Entrada (Lexing)**

**\*\*Entrada\*\***: Cadena a analizar, mapa de regex compiladas

**\*\*Salida\*\***: Array de tokens identificados o errores léxicos

**\*\*Algoritmo:\*\***

...

posición := 0

tokens := []

errores := []

Mientras posición < longitud(entrada):

Si entrada[posición] es espacio:

Salta espacios

Continúa

mejorCoincidencia := null

mejorLongitud := 0

mejorTerminal := null

Para cada terminal en terminales:

Intenta match en posición actual

Si coincide y longitud > mejorLongitud:

mejorCoincidencia := match

mejorLongitud := lenght(match)

mejorTerminal := terminal

Si mejorCoincidencia:

tokens.push({

type: mejorTerminal.name,

lexeme: mejorCoincidencia,

line: línea\_actual,

column: columna\_actual

})

posición += mejorLongitud

Si no:

errores.push("No se puede tokenizar en posición X")

posición += 1

Retorna {

ok: errores.length === 0,



```
tokens,
errors: errores
}
...
```

**\*\*Ejemplo:\*\***

...

Entrada: "(a+a)FIN"

Terminales: {

```
$_paren_open: /\(\(/ ,
$_a: /^a/ ,
$_plus: /\+\+/,
$_paren_close: /\)\)/ ,
$_fin: /^FIN/
}
```

Resultado: [

```
{ type: '$_paren_open', lexeme: '(' },
{ type: '$_a', lexeme: 'a' },
{ type: '$_plus', lexeme: '+' },
{ type: '$_a', lexeme: 'a' },
{ type: '$_paren_close', lexeme: ')' },
{ type: '$_fin', lexeme: 'FIN' }
]
...
```

## II. Parser LL(1)

### 1. Cálculo de Conjuntos FIRST

**\*\*Definición\*\*:**  $\text{FIRST}(\alpha)$  = conjunto de terminales que pueden aparecer primero en una derivación de  $\alpha$

**\*\*Algoritmo (Fixed-Point Iteration):\*\***

...

Para cada no-terminal A:

$\text{FIRST}(A) := \emptyset$

Repetir hasta convergencia:

Para cada producción  $A \rightarrow X_1 X_2 \dots X_n$ :

Para  $i := 1$  hasta  $n$ :

$\text{FIRST}(A) := \text{FIRST}(A) \cup \text{FIRST}(X_i) - \{\epsilon\}$  //  $\epsilon$  no existe en este sistema

Si  $X_i$  no puede derivar en  $\epsilon$ :

break

...

**\*\*En contexto Wilson (sin epsilon):\*\***

```javascript

function buildFirstSets(terminals, nonTerminals, productions) {

const firstSets = {};

```
// Inicializa
nonTerminals.forEach(nt => {
    firstSets[nt] = new Set();
});

// Fixed-point iteration
let changed = true;
while (changed) {
    changed = false;

    for (const [nonTerminal, alternatives] of Object.entries(productions)) {
        for (const alternative of alternatives) {
            const symbols = parseAlternative(alternative);

            for (const symbol of symbols) {
                if (isTerminal(symbol)) {
                    // Terminal: añade directamente
                    if (!firstSets[nonTerminal].has(symbol)) {
                        firstSets[nonTerminal].add(symbol);
                        changed = true;
                    }
                    break; // No epsilon en este sistema
                } else {
                    // No-terminal: añade su FIRST
                    const beforeSize = firstSets[nonTerminal].size;
                    firstSets[symbol].forEach(t =>
```

```

        firstSets[nonTerminal].add(t)

    );

    if (firstSets[nonTerminal].size > beforeSize) {

        changed = true;

    }

    break; // No epsilon

}

}

}

}

}

```

```

return firstSets;

```

```

}

```

```

...

```

```

**Ejemplo con a+a:**

```

```

...

```

Producciones:

```

%_E <= '(' %_R ')' 'FIN'

```

```

%_R <= $_a '+' %_R | $_a

```

```

FIRST(%_E) = { '(' }

```

```

FIRST(%_R) = { $_a }

```

```

...

```

2. Parser LL(1) Predictivo

****Estructura de datos:****

- ****Pila (Stack)****: Contiene símbolos y referencias a nodos del árbol
- ****Entrada****: Array de tokens
- ****Lookahead****: Token actual (1 símbolo de anticipación)

****Algoritmo Principal:****

...

stack := [\$, S] // \$ = final, S = símbolo inicial

input := tokens + [\$]

entrada_pos := 0

derivationTree := crear_nodo(S)

errors := []

Mientras stack not empty:

 top := stack.pop()

 lookahead := input[entrada_pos]

 Si top es terminal:

 Si top === lookahead.type:

 Crea nodo hoja con lexema

 entrada_pos := entrada_pos + 1

 Si no:

 errors.push("Error de coincidencia esperado " + top)

break

Si top es no-terminal:

// Selecciona producción via FIRST set

alternativa := selecciona_producción(top, lookahead, FIRST)

Si no hay alternativa:

errors.push("Error sintáctico, no hay producción para " + top)

break

Si no:

// Expande: crea nodos hijos y agrega símbolos a stack

nodos_hijos := []

Para cada símbolo en alternativa (inverso):

nodo_hijo := crear_nodo(símbolo)

nodos_hijos.push(nodo_hijo)

stack.push(símbolo)

Asigna nodos_hijos a top.children

Si entrada_pos === length(input)-1 y stack empty:

Retorna { ok: true, tree: derivationTree }

Si no:

Retorna { ok: false, errors: errors }

...

****Selección de Producción:****

...

Para cada alternativa de no-terminal:

 firstSymbols := FIRST(alternativa)

 Si lookahead.type in firstSymbols:

 Retorna alternativa

Retorna null (error)

...

****Ejemplo con (a+a)FIN:****

...

Paso 1:

Stack: [\$, %_E]

Lookahead: \$_paren_open '('

FIRST(%_E) = { '(' }

Producción: %_E <= '(' %_R ')' 'FIN'

Stack: [\$, '\$_fin', '\$_paren_close', %_R, '\$_paren_open']

Paso 2:

Top: \$_paren_open

Lookahead: \$_paren_open '('

Coincide, consume

Stack: [\$, '\$_fin', '\$_paren_close', %_R]

Paso 3:

Top: %_R

Lookahead: \$_a 'a'

FIRST(%_R) = { \$_a }

Producción: %_R ≤ \$_a '+' %_R | \$_a

Elige primera alternativa (ambas empiezan con \$_a, pero primera es más larga)

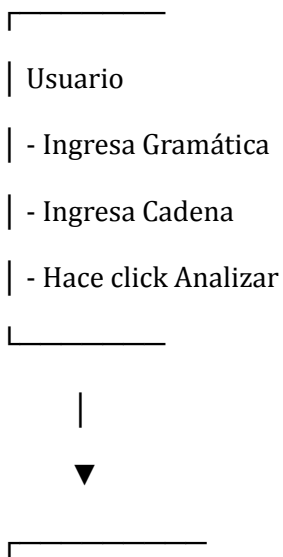
Stack: [\$, '\$_fin', '\$_paren_close', %_R, '\$_plus', \$_a]

... (continúa)

...

Flujo de Datos Completo

Fase 1: Usuario Ingresa Datos



- | Frontend (Angular)
 - | - Valida campos no vacíos
 - | - Prepara JSON payload
 - | - POST a localhost:3000/analyze
-

Fase 2: Backend Recibe Request

...

-
- | Endpoint: POST /analyze
 - | Body: {source, input}
 - | Validación:
 - | - source $\neq$ ""
 - | - input es string
-

|



control_ok

Fase 3: Análisis de Gramática

-
- | parseWisonWithJison(source)
 - | - Pasa a Jison meta-parser
 - | - Extrae bloques Lex y Syntax

- | - Ejecuta extractTerminals()
- | - Ejecuta extractProductions()
- | - Ejecuta extractStartSymbol()

|



validateWisonGrammar()

|— Líneas sintácticamente correctas

|— Detecta recursión izquierda

|— Verifica conflictos FIRST/FIRST

|— Valida referencias de símbolos

|— Calcula conjuntos FIRST

|



Si hay errores —► Retorna 400 {grammarErrors}

Si no —————► Continúa

...

Fase 4: Análisis Léxico

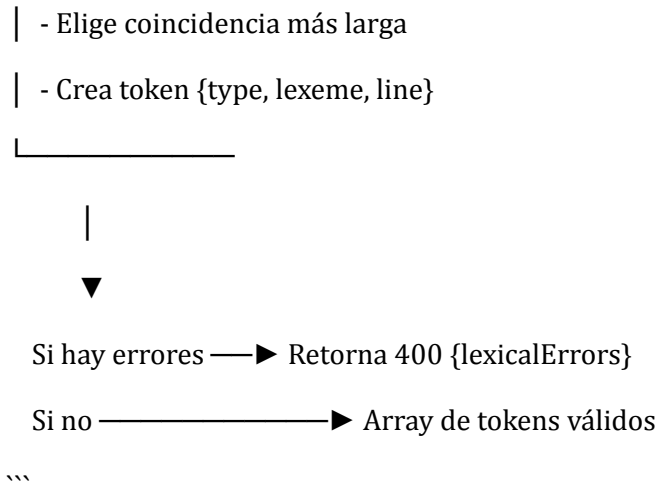
...

| tokenizeInput(input, terminals)

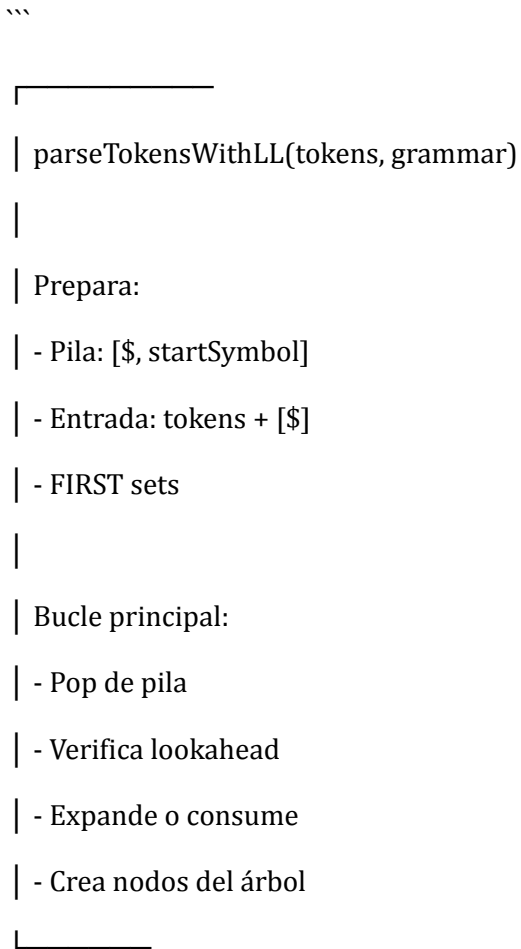
|

| Para cada posición en input:

| - Intenta match con cada terminal



Fase 5: Análisis Sintáctico



|



Si errores $\longrightarrow$ { ok: false, errors }

Si éxito $\longrightarrow$ { ok: true, derivationTree }

...

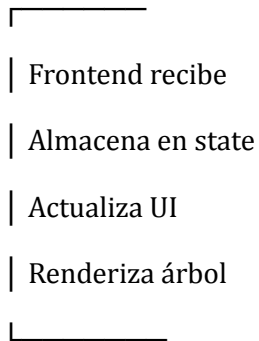
Fase 6: Respuesta HTTP

...

```
|  
|  
| res.json({  
|   ok: true,  
|   grammar: {  
|     ok, terminals, nonTerminals,  
|     productions, startSymbol, errors  
|   },  
|   lexical: {  
|     ok, tokens, errors  
|   },  
|   syntax: {  
|     ok, derivationTree, errors, steps  
|   }  
| })  
|
```

|





Limitaciones y Consideraciones

Limitaciones de Diseño (Intencionadas)

1. **Sin producciones epsilon (vacías)**

- Simplifica significativamente el cálculo de FIRST/FOLLOW
- Una alternativa siempre debe contener al menos un símbolo

$%_A \leq | \%_B ; \quad // \text{ No permitido}$

$%_A \leq \%_B ; \quad // \text{ Correcto}$

2. **Sin recursión izquierda**

- El parser LL(1) no puede manejar recursión izquierda
- Debe transformarse a recursión derecha

$\%_E \leq \%_E '+' \%_T | \%_T ;$

$\%_E \leq \%_T '+' \%_E | \%_T ;$

3. ****LL(1) únicamente****

- Un símbolo de lookahead
- Si dos alternativas comienzan con el mismo terminal → Error

`%_S <= 'a' 'b' | 'a' 'c' ; // FIRST/FIRST conflict`

`%_S <= 'ab' | 'ac' ; // Resuelto`

4. ****Sin acciones semánticas****

- El sistema solo construye el árbol de derivación
- No ejecuta código durante el análisis
- No hay tokens de atributos

5. ****Almacenamiento en memoria****

- Los árboles se generan completamente en memoria
- Grammars muy grandes (>1000 producciones) pueden ser lentas

Consideraciones de Rendimiento

| Aspecto | Límite Recomendado | Razón |

|-----|-----|-----|

| Tamaño de gramática | < 100 producciones | Cálculo FIRST/FOLLOW |

| Tamaño de entrada | < 10,000 caracteres | Tokenización greedy |

| Profundidad de árbol | < 100 niveles | Stack del parser |

| Alternativas por no-terminal | < 20 | Selección lineal de producción |

Limitaciones Técnicas

1. ****Jison meta-parser****

- Basado en bison/yacc
- Parsea una sola vez al startup del backend
- No se puede cambiar dinámicamente la gramática de Wison

2. **Expresiones regulares**

- Limitadas a JS regex syntax
- No soporta lookahead/lookbehind
- No soporta cuantificadores {n,m}

3. **Frontend**

- Localhost:4200 (desarrollo) o producción
- Textarea: máximo ~100,000 caracteres
- Árbol ASCII: máximo ~10,000 líneas visibles

Guía de Instalación

Requisitos Previos

- **Node.js**: 18 LTS o superior
- **npm**: 10.x o superior
- **Git**: Para clonar el repositorio
- **Editor**: VS Code recomendado

Instalación Paso a Paso

1. Clonar Repositorio

powershell

```
cd "C:\ruta\del\proyecto"
```

```
git clone <repo-url>
```

```
cd compi_practica2
```

2. Instalar Backend

powershell

```
cd backend
```

```
npm install
```

Verifica las dependencias

```
npm list --depth=0
```

****Salida esperada:****

```
backend@1.0.0 C:\...\backend
```

```
├── cors@2.8.6
```

```
├── express@5.2.1
```

```
└── jison@0.4.18
```

```
...
```

3. Instalar Frontend

powershell

```
cd ..\frontend
```

```
npm install
```

Verifica las dependencias

```
npm list --depth=0
```


****Salida esperada:****

@angular/common@21.2.0

@angular/compiler@21.2.0

@angular/core@21.2.0

@angular/forms@21.2.0

@angular/platform-browser@21.2.0

@angular/router@21.2.0

rxjs@7.8.0

tslib@2.3.0

typescript@5.9.2

4. Verificar Encoding

Asegúrate de que todos los archivos estén en ****UTF-8 sin BOM****:

powershell

En VS Code:

- Abre Status Bar (parte inferior)
- Busca "UTF-8"
- Click y selecciona "UTF-8"
- Asegúrate de que NO diga "UTF-8 with BOM"

O usa .editorconfig (ya incluido):

ini

[*]

charset = utf-8

```
end_of_line = lf
```

```
insert_final_newline = true
```

5. Configuración de Proxy (Frontend)

El archivo `frontend/proxy.conf.json` ya está configurado:

```
json
```

```
{
```

```
  "/analyze": {
```

```
    "target": "http://localhost:3000",
```

```
    "pathRewrite": {"^/analyze": "/analyze"}
```

```
  },
```

```
  "/tokenize": {
```

```
    "target": "http://localhost:3000",
```

```
    "pathRewrite": {"^/tokenize": "/tokenize"}
```

```
  },
```

```
  "/generate": {
```

```
    "target": "http://localhost:3000",
```

```
    "pathRewrite": {"^/generate": "/generate"}
```

```
  },
```

```
  "/health": {
```

```
    "target": "http://localhost:3000",
```

```
    "pathRewrite": {"^/health": "/health"}
```

```
  }
```

```
}
```

```
...
```

Instalación Completada

Si no hay errores, la instalación está lista. Procede a [Guía de Ejecución](#guía-de-ejecución).

Guía de Ejecución

Inicio del Backend

powershell

En terminal 1

cd backend

npm start

Salida esperada:

Wison Backend ejecutándose en puerto 3000

(espera en <http://localhost:3000>)

****Verificar que corre:****

powershell

En otra terminal

curl <http://localhost:3000/health>

Respuesta esperada:

```
{"ok":true,"service":"wison-backend"}
```

Inicio del Frontend

powershell

En terminal 2 (diferente)

cd frontend

npm start

Salida esperada:

build completed successfully

ng serve is listening on localhost:4200

Open your browser on <http://localhost:4200>

Acceder a la Aplicación

1. Abre navegador
2. Ve a `http://localhost:4200`
3. Deberías ver la interfaz Wison

****Interfaz esperada:****

- Header: "Analizador Wison"
- Textarea con gramática a+a por defecto
- Campo de entrada con "(a+a+a)FIN"
- Botón azul "Analizar"
- Sección de resultados vacía

Hacer tu Primer Análisis

1. Click en botón "Analizar"
2. Espera unos segundos (los primeros análisis son más lentos)

3. Deberías ver:

- Estado: "Análisis completado"
- Árbol de derivación ASCII
- Información de estructura en secciones colapsables

****Árbol esperado:****

...

%_E

| - '(' %_paren_open

 | - %_R

 | | - \$_a

 | | | - '+' \$_plus

 | | | - %_R

 | | | - \$_a

 | - ')' \$_paren_close

 | - 'FIN' \$_fin

...

Detención

powershell

Backend: Presiona Ctrl+C en terminal 1

Frontend: Presiona Ctrl+C en terminal 2

Ejemplos de Uso

Ejemplo 1: Expresión Aritmética Simple (a+b)

****Gramática Wison:****

```
``wison
```

```
Wison¿
```

```
Lex {
```

```
    Terminal $_plus <- '+' ;
```

```
    Terminal $_minus <- '-' ;
```

```
    Terminal $_a <- 'a' ;
```

```
    Terminal $_b <- 'b' ;
```

```
}
```

```
Syntax {
```

```
    No_Terminal %_E ;
```

```
    No_Terminal %_T ;
```

```
    Initial_Sim %_E ;
```

```
    %_E <= %_T | %_E '+' %_T | %_E '-' %_T ;
```

```
    %_T <= $_a | $_b ;
```

```
}
```

```
?Wison
```

```
...
```

```
**Entrada de prueba:**
```

```
...
```

```
a+b+a
```

```
...
```

```
**Resultado esperado:**
```

```
- Estado: Análisis completado
```

- Árbol mostrará: $a + b + a$ con estructura de derivación

Ejemplo 2: Identificadores Simples

****Gramática:****

``wison

Wison

Lex {{

Terminal \$_letter <- '[a-zA-Z]';

Terminal \$_digit <- '[0-9]';

}}

Syntax {{

No_Terminal %_ID ;

Initial_Sim %_ID ;

%_ID <= \$_letter | %_ID \$_letter | %_ID \$_digit ;

}}

FIN

...

****Entrada:****

...

abc123

...

****Resultado:****

- Identificador válido
- Árbol con estructura de derivación rightshifted

Ejemplo 3: Gramática con Error (Recursión Izquierda)

****Gramática INCORRECTA:****

``wison

Wison;

Lex {

Terminal \$_plus <- '+' ;

Terminal \$_num <- '[0-9]' ;

}

Syntax {

Initial_Sim %_E ;

%_E <= %_E '+' \$_num | \$_num ; // Recursión izquierda

}

?Wison

...

****Interior de envío:****

...

1+2+3

...

****Resultado esperado:****

- Error: "Recurción izquierda detectada: %_E"
- No continúa al análisis léxico

Ejemplo 4: Paréntesis Anidados

****Gramática:****

``wison

Wison;

Lex {

Terminal \$_open <- '\\(' ;

Terminal \$_close <- '\\)' ;

}

Syntax {

No_Terminal %_S ;

Initial_Sim %_S ;

%_S <= \$_open %_S \$_close | \$_open \$_close ;

}

?Wison

...

****Entradas válidas:****

- `0`

- `(0)`

- `((0))`

****Entrada inválida:****

- `00`

- `)))C

Conclusión

Este manual cubre todos los aspectos técnicos del proyecto Wison: desde la arquitectura general hasta los detalles de implementación de cada componente, algoritmos, y instrucciones de uso práctico.

Para preguntas adicionales o reportes de errores, consulta el repositorio del proyecto o contacta al equipo de desarrollo.

****¡Feliz análisis!****

Última actualización: Abril 2026

Versión del documento: 1.0